CryptAlyst — Books, Models & Data

The decoupled streaming forecast architecture

> *A complete review of the architecture designed and built in this latest development:* ***messaging, streaming, models, books, data and history*** *— and why they must be combined to continuously forecast and fire* ***tier signals****.*

---

1. The core principle

The system is **asynchronous and decoupled**. Nothing calls anything else directly.

- **Models are calculation agents** — they publish *numbers* (a signal, a score, a probability).
- **The bus carries packets** — every message is an envelope with an **epoch timestamp and a TTL**, and it is **never backlogged**.
- **Books hold the intel** — a book accumulates the recent signal stream in a **bounded rolling window**, so a model can see a *pattern* before it is confident.
- **Strategies trade** — a strategy lives inside the book and opens/closes only when the model has confirmed a pattern.

A single signal is not enough. A model reports a signal, is *not sure* whether it is a good idea, and waits until it sees a **pattern**. **That intel is what the book carries and the model uses.**

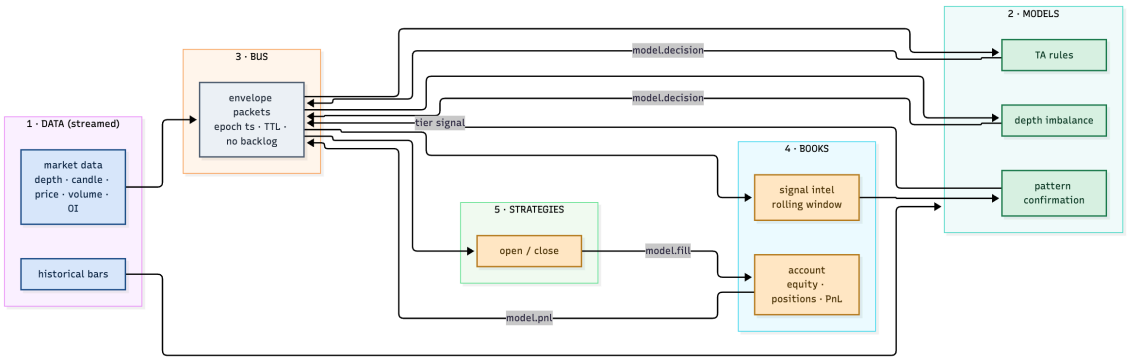


Figure 1.1 — data → models → bus → books → strategy → trade, decoupled at every hop.

---

2. Messaging — the bus is the nervous system

Every message on the bus is an **envelope**: a payload plus metadata — the provenance that makes the stream auditable and fresh.

Field	Meaning
`topic`	the channel (e.g. `model.decision`, `market.depth`)
`producer`	who may write this topic (enforced by the topic registry)
`epoch timestamp`	when it happened (`computed_at`, `received_at`, UTC)
`generation` / `seq`	monotonic ordering per topic

`TTL / retention`	how long the broker keeps it (`drop`, `60s`, `5m`)
`cid` / `parent_cid`	correlation id — links a decision to its fills
`hops` / `refs`	the path the packet travelled and the data it references

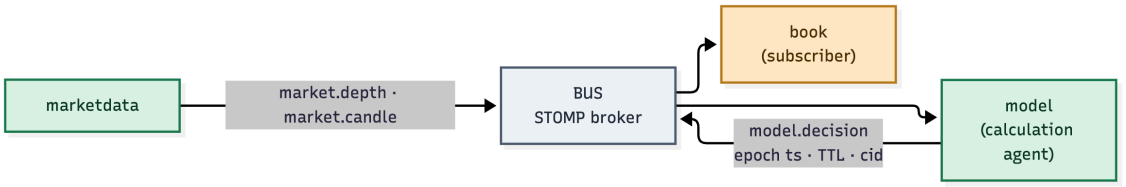


Figure 2.1 — calculation agents publish numbers; subscribers listen and decide.

2.1 Topics — a single registry with producer contracts

`topics.go` is the **one table**. Each topic declares its exchange, its producers and its retention (TTL).

Topic	Exchange	Producers	Retention
`market.price` / `market.depth` / `market.candle`	market	market_data	drop (live only)
`intel.signal` / `intel.flow` / `intel.movers`	intel	signals	60s
`intel.ta`	intel	tad	60s
`intel.forecast`	intel	forecast	60s
`model.decision`	model	models10, curve_rider, …	5m
`model.fill`	model	same	5m
`model.pnl`	model	same + supervisor	5m
`risk.state`	risk	risk	5m

Why TTL + no backlog: a market moves on the *latest* information. If packets queued, a slow subscriber would act on stale data. With TTL the bus is a **state channel**, not a job queue — subscribers read the current picture, and old packets evaporate.

---

3. Streaming — never poll (BPF-inspired)

The Berkeley Packet Filter filters packets **at the source**, so only the data that matches is ever copied up. The same idea applied to market data:

1. **Filter at the source** — subscribe to only what we need (`depth5` , not the full book).
2. **Keep only the signal** — collapse the order book to one `imbalance` number.
3. **Never poll** — data pushes to us; decisions happen on arrival.

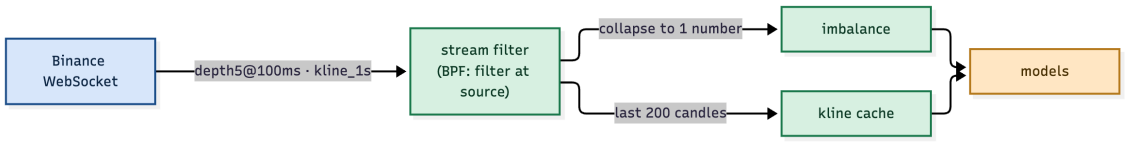


Figure 3.1 — WebSocket → filter → one number, the BPF pattern.

Measured during the build: **one model calculation takes ~150 µs**, while **one HTTP fetch takes ~333 ms** — the network was ~2000× slower than the math. Polling 24 endpoints sequentially cost ~8 seconds per decision; the stream delivers the same data in real time. **The bottleneck was never the language — it was I/O and polling.**

---

4. Models — calculation agents that publish numbers

A model owns **one question**. It does not know about accounts, positions, or risk. It computes and publishes — which is what makes it replaceable and testable.

Model	Question	Output
`donchian55`	breaking a 55-bar range?	buy/sell
`orb`	breaking the opening range?	buy/sell
`boll_squeeze`	tight Bollinger expanding?	buy/sell
`rsi2`	2-bar RSI overbought/oversold?	buy/sell
`vwap_revert`	stretched from VWAP?	buy/sell
`macd_trend`	MACD crossed zero?	buy/sell
`vol_break`	volume spike + directional close?	buy/sell
`ema_pullback`	pullback to the EMA trend?	buy/sell
`atr_channel`	outside an ATR channel?	buy/sell
`curve_rider`	strongest book imbalance?	buy/sell + score

4.1 Models feed other models

A model's output is input to another. A **positioning** model publishes where the book is exposed; a **risk** model consumes it and publishes risk; both feed the decision. **Models provide valuable data for other models** — and the bus is how that data moves.

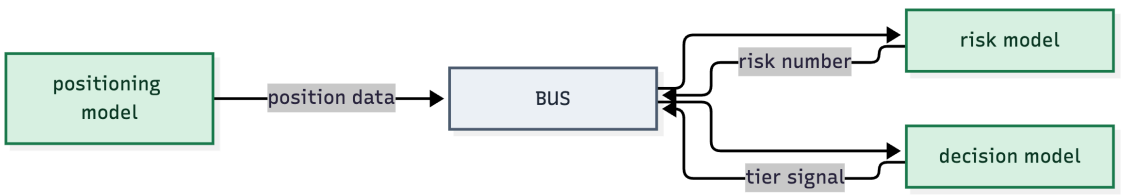


Figure 4.1 — positioning → risk → decision, each a published number.

---

5. Books — account and intel

The book is two things at once:

1. **The account** — equity, open positions, realised PnL.
2. **The intel** — a bounded rolling window of recent signals, so the model can see a *pattern* rather than a lone blip.

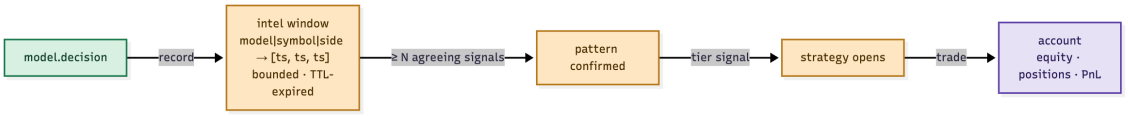


Figure 5.1 — a lone signal is noise; the streak in the window is the signal.

"Never hold too much data, but imagine a bull run." The window is bounded — old entries expire — yet it is long enough that during a sustained move the model sees the *streak* of agreeing signals that confirms the trend.

---

6. Data — the market microstructure

A forecast is only as good as the data behind it. The system reads the order flow, not just the candles:

Signal	Reads	What it reveals
depth imbalance	bid vs ask volume	real buy/sell pressure now
OI change	open interest over an interval	new money entering
price	last / 24h move	direction + momentum
volume	quote volume / spike	participation and conviction

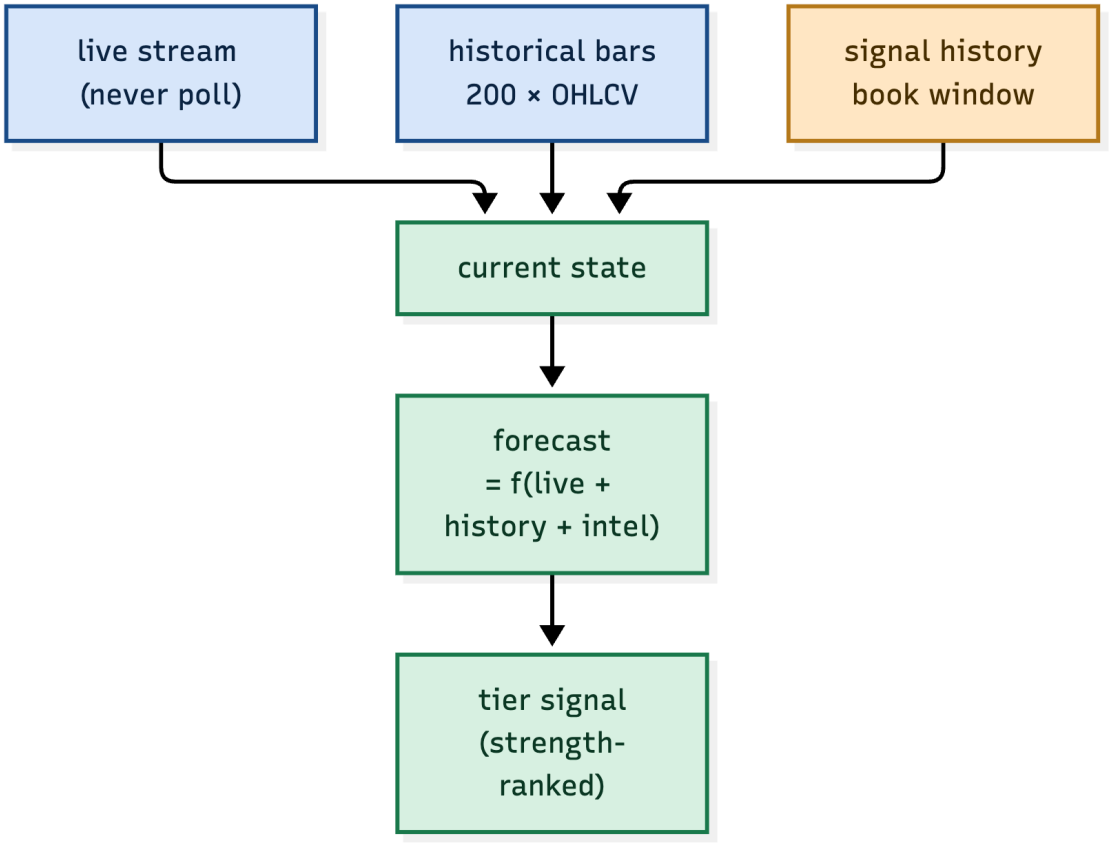


Figure 6.1 — three layers combined turn a number into a forecast.

All three layers combined turn a number into a *forecast*, and a forecast into a **tier signal** — a confidence-ranked decision the strategy acts on.

---

7. From forecast to tier signal to trade

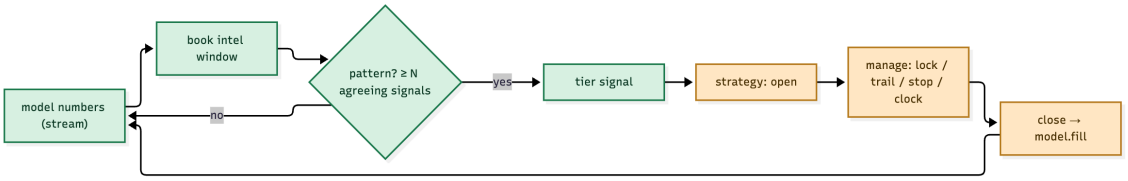


Figure 7.1 — numbers → pattern → tier signal → open → manage → fill.

The **exit ladder** (what the strategy manages):

Exit	Rule	Purpose
`lock`	+0.35%	bank a fee-clearing profit
`trail`	peak ≥ +1%, exit −0.5% off peak	ride winners

`stop`	−0.5%	cap every loss (no heavy bleeds)
`clock`	180s hold, winners only	force turnover

---

8. Pure Go — async and concurrent

The models and the engine run as **Go goroutines**, each decoupled from the others: a slow model can't block the others, a panicking model can't kill the pipeline, and models run continuously whether or not anyone listens. This is the runtime mirror of the bus: the same independence the bus gives at the message level, goroutines give at the process level.

---

9. Why this is a "tier signal" system

- **A model alone is an opinion; a book with history is a conviction.**
- **Data alone is noise; data + context + memory is a forecast.**
- **The bus keeps it decoupled and real-time; TTL keeps it fresh; the window keeps it informed.**

That combination — *combined, historical, and continuously updated* — is what lets the system **forecast** and then **fire a tier signal** a strategy can trade with confidence.